Document 04 — OpenAI API → Project 1

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

01, 02, 03 — you'll reuse two Doc01 scripts here, copied unchanged into `practice/build_task/`:

- `config.py` — **What:** `load_config() -> Config`. **Why:** every script in this app needs `OPENAI_API_KEY` loaded the same way. **How:** copy `01_python_foundations/practice/build_task/config.py` in as-is; don't rewrite it.
- `logging_setup.py` — **What:** `get_logger(name)`. **Why:** consistent logging across this app, same as every other document. **How:** copy `01_python_foundations/practice/build_task/logging_setup.py` in as-is; don't rewrite it.

Doc03's token/cost/context-limit intuition and Doc02's retry pattern also get used for real here — see each exercise's **Builds on** line below.

How to Read & Practice This Document

- **What:** calling OpenAI's API directly, with no extra library.
- **Why:** every library you learn later (LangChain, LangGraph) is really just a wrapper around this. To debug them later, you need to know what's underneath.
- **When:** any time you build or fix an LLM feature — library or not, this is the layer everything else sits on top of.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercises closed-book. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open. 4. Try **Project 1** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1 → Hint 4** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud how structured output actually gets enforced under the hood. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Everything so far has been preparation. Doc01 gave you a config loader and a logger. Doc02 gave you a disciplined way to make HTTP calls. Doc03 gave you a working mental model of what happens inside a call to a model. This document puts all three to work on a real, live OpenAI call — and ends with your first real project.

First, the `openai` library is really just Doc02's HTTP layer wrapped in a nicer shape — `client.chat.completions.create(...)` sends the `messages` list (Doc03's "memory" idea, made concrete) and gets a reply back.

Second, a reply can come back all at once, or **streamed** piece by piece as it's written — the same total wait, but a very different feel for a user watching it.

Third, when your code needs to actually *use* the model's answer (not just show it to someone), asking nicely for JSON in a prompt isn't good enough — **structured outputs** forces the model's answer into a shape you define, so you get back a checked object or a clear failure, never a guess.

Fourth, the library's errors map directly onto Doc02's retry rules — `AuthenticationError` and `BadRequestError` are permanent (fix the request, don't retry), `RateLimitError` and `APITimeoutError` are temporary (back off and retry) — so everything you learned in Doc02 about *which* failures to retry applies here without any new rules to learn.

Fifth, there are two APIs for this — Chat Completions (the steady, foundational one you're learning here) and the newer Responses API (which manages conversation history for you, better suited to agent-heavy work later). You start with Chat Completions because it's what almost everything else, including most of LangChain, is actually built on underneath.

That's the story: this document turns three documents' worth of preparation into one real, working call to a real model — and the **Build Task** (Project 1) asks you to put a full small app around it: a chatbot with real memory, streaming replies, structured output, and graceful handling of the failures Doc02 and Doc03 already taught you to expect.

Core Concepts (read this first — everything you need is here)

Topics on this page: The client, and the list of messages · Streaming vs. waiting for the full reply · Structured output: how "give me JSON" is actually guaranteed · Error types, and which Doc02 rules apply to each · Chat Completions vs. the newer Responses API

The client, and the list of messages

The `openai` library gives you a **client** — one Python object that holds your API key and sends the HTTP requests for you, the same kind of request/response call Doc02 taught you to build by hand. You create it once, `client = OpenAI()`, then call `client.chat.completions.create(model=..., messages=[...])`. The `messages` list **is** the whole conversation. Think of the model as a stranger with no memory who only ever sees what you

hand them right now — every time you want a reply, you hand over the whole transcript again, plus your new line, because Doc03's "no memory" rule is exactly what is happening here.

How it really works

- `OpenAI()` reads `OPENAI_API_KEY` straight from the environment — so `load_dotenv()` (Doc01's `load_config()` pattern) must run before you create the client, or the key isn't there yet.
- Each entry in `messages` is a small dict with a `role` (`system` / `developer`, `user`, `assistant`, `tool`) and `content`. `system` or `developer` holds your standing rules; put it once, at index 0. `tool` carries a tool's result back to the model — full details in Doc06.
- The client keeps its own connection pool and its own default timeout/retry settings, doing under the hood roughly what Doc02's `http_client.py` does by hand — set them on purpose: `OpenAI(timeout=30.0, max_retries=3)`.
- Memory is only what you resend. Skip appending the assistant's reply and the model "forgets" what it just said on the next turn; never trim the list and it eventually hits the context limit (Doc03).
- `response.choices[0].message.content` is the text; `response.usage` is the token count you log (Doc01's `get_logger`, Doc03's cost topic) and pay for; `finish_reason` tells you why the model stopped — `"stop"` is normal, `"length"` means it was cut off by `max_tokens`.
- Making a new `OpenAI()` client inside a function or a loop still works, but it opens fresh connections every call — create one, near the top of a file, and reuse it.

Situation	What to do	Why
Your app starts	Create **one** client and reuse it everywhere	A client keeps a connection pool; a new one per call is slower and can run out of connections
Multi-turn chat	Append user → call → append assistant reply, every turn	This is the model's only kind of memory (Doc03)
A one-off task (summarize one email)	Build a fresh 2-message list, no history	Old turns cost tokens and add nothing
Changing how the model behaves	Edit the `system` / `developer` message, not the user turns	Keeps "rules" separate from "conversation"
Conversation getting long	Keep the system message, drop or summarize the oldest turns	Stops the context-limit error before it happens (Doc03)

Common mistakes:

- *Mistake:* forgetting to append the assistant's reply back into `history`. → *Symptom:* the model repeats itself or "forgets" what it just said, one turn later. → *Fix:* append both the `user` message

and the `assistant` reply, every turn — no exceptions.

- *Mistake:* creating `OpenAI()` inside the function that handles each request. → *Symptom:* calls get slower under load, and a busy server can run out of connections. → *Fix:* create the client once, at module level or in one `create_client()`, never inside a loop.

Where you'll meet it: Project 1 (SupportDesk chat) is this exact list, kept alive across a whole conversation. In Doc05, LangChain's `ChatOpenAI` builds this same list with `SystemMessage` / `HumanMessage` / `AIMessage`. Doc06 adds `tool` messages to it. Doc09 keeps this list inside the graph's state. In Doc11 and Project 4, each agent keeps its own `messages` list with its own system prompt — a multi-agent system is, underneath, several of these lists plus code that passes results between them.

Quick cheat sheet:

- One client, created once, reused everywhere. Key comes from `OPENAI_API_KEY`, loaded the Doc01 way — never hardcoded.
- The `messages` list is the whole memory — resend all of it, every call.
- Always append the assistant's reply back into the list.
- Read `finish_reason`; log `response.usage` with Doc01's logger — that is your cost record (Doc03).

Streaming vs. waiting for the full reply

A normal call waits until the model finishes writing, then hands you the whole answer at once. A **streaming** call (`stream=True`) sends the answer back in small pieces called **chunks**, while the model is still writing — like watching someone type instead of waiting for them to hand you a finished letter. The total time to finish is the same either way; streaming only changes *when* the user starts seeing something, and that "time to first token" is the number people actually feel.

How it really works

- Each chunk carries `delta` (the new piece), never `message` (the full text) — you join the pieces yourself, nothing is saved for you.
- `delta.content` can be `None` on some chunks (often the first and last) — always use `or ""`, or `"".join(...)` crashes with `TypeError`.
- Ask for `stream_options={"include_usage": True}` to still get usage totals (Doc03's cost topic) — they arrive on one final chunk whose `choices` list is **empty**, so check `if chunk.choices:` before indexing `[0]`, or you get an `IndexError` right at the end of an otherwise-fine answer.
- A connection can drop mid-stream, raising `openai.APIConnectionError` **during** the `for` loop, not only at the `create()` call — the `try` has to wrap the whole loop, not just the call that starts it. This is Doc02's timeout/connection-error territory, just arriving mid-response instead of up front.

- Memory still needs the **full** joined text: append the complete joined string to `history` as the assistant turn, same rule as the topic above.
- Don't stream when your code needs the whole answer to act on it (structured output, a single-word label) — half a JSON object is useless, so you wait for the end anyway.

Situation	What to do	Why
A person is watching a chat window	Stream	First words appear fast; the app feels alive
Code needs the full answer before using it (JSON, a label)	Don't stream	Half an answer is useless; you wait for the end anyway
A background job (nightly report, batch of emails)	Don't stream	Nobody is watching; streaming only adds code
A web API passing replies to a browser	Stream from OpenAI, pass chunks on	Same "typing" feel for the browser user (Doc12)
You still need token totals	Add `stream_options={"include_usage": True}`	Streamed chunks carry no usage unless asked

Common mistakes:

- *Mistake:* joining `delta.content` with no `or ""` guard. → *Symptom:* `TypeError: sequence item X: expected str instance, NoneType found`, on a call that "looked fine" moments before. → *Fix:* `chunk.choices[0].delta.content or ""` on every chunk.
- *Mistake:* reading `chunk.choices[0]` after turning on `include_usage`, with no check. → *Symptom:* `IndexError`, right after the whole reply already printed correctly. → *Fix:* `if chunk.choices:` before indexing — the usage chunk's `choices` list is empty by design.

Where you'll meet it: Project 1 streams its replies in the terminal. In Doc05, LangChain's `.stream()` is this same loop underneath. In Doc09 and Doc11, a multi-agent graph streams progress events so a user sees which agent is working instead of one long silent wait. In Doc12, once your agent team is a FastAPI service, these same chunks pass straight through to whoever called it.

Quick cheat sheet:

- Streaming changes *when* the user sees words, not the total time.
- Stream for people watching; don't stream for code that needs the complete answer.
- `delta.content` can be `None` — always `or ""`.
- Wrap the whole `for` loop in `try` — a stream can break mid-way, not just at the start.
- Join the pieces and append the full text to `history`, or memory breaks.

Structured output: how "give me JSON" is actually guaranteed

Asking the model in a prompt to "answer in JSON" is only a request — it can still write broken JSON, add a sentence before it, or rename a field. **Structured outputs** is stronger: you give the API a schema (usually a Pydantic model), and it limits which tokens the model is allowed to write, a technique called **constrained decoding** — any token that would break the schema simply isn't offered. It's like a form with locked fields instead of a blank page: the model can only fill in the blanks you defined, in the type you defined, so your code gets back a checked object or a clear failure, never a guess.

How it really works

- Three layers, each stronger than the last: a plain prompt guarantees nothing; `response_format={"type": "json_object"}` (JSON mode) guarantees valid JSON but not the right keys; structured outputs with `strict` on guarantees the right keys, types, and enum values.
- The shape is guaranteed; the **truth** is not — the model can still pick the wrong category or invent a date that matches the schema perfectly and is still wrong. Check important values yourself, the same "don't trust a 200 blindly" instinct Doc02 taught for HTTP JSON.
- `client.beta.chat.completions.parse(..., response_format=MyModel)` does three jobs at once: sends your model as a strict schema, reads the JSON back, and runs Pydantic's `model_validate` on it — automating the manual shape-check Doc02's JSON topic walked through by hand.
- Always check `message.refusal` before touching `message.parsed` — the model can decline (for safety reasons) instead of returning either.
- If `max_tokens` is too small, the model's JSON gets cut off mid-object — `finish_reason == "length"` (Doc03's context-window topic) and `.parse()` raises `openai.LengthFinishReasonError`.
- Making a field required (`order_id: str`) when the user never gave one forces the model to invent a value just to satisfy the schema — allow `X | None` and check for it instead.
- A schema can't know business rules ("this date can't be in the past") — add a Pydantic `field_validator` for those.

Situation	What to do	Why
Pull fields out of free text	Structured output with a Pydantic model	A missing key would crash code that reads it directly
Sort into fixed categories	A `Literal[...]` field	The model can only pick one of your exact values
A supervisor picks the next agent (Doc11)	Structured output, agent names as a `Literal`	A typo'd agent name would break routing
The model might not have an answer	Allow `str | None` , check `message.refusal`	Forcing a value makes the model invent one

Situation	What to do	Why
The answer is shown only to a person	Don't use it — plain text	Nothing in code reads it; the schema only adds limits

Common mistakes:

- *Mistake:* making a field required when the value might genuinely be missing. → *Symptom:* neat, valid-looking data with an invented `order_id` the user never gave. → *Fix:* `X | None` for anything optional, and check for `None` in your own code.
- *Mistake:* treating a schema-valid object as a correct one. → *Symptom:* `category` is a real value from your `Literal`, just the wrong one for this ticket. → *Fix:* validate against real examples and business rules — the schema checks shape, Doc13 checks correctness.

Where you'll meet it: Project 1's "pull out the fields" mode uses this directly. Doc06's tool arguments are the same idea — a schema the model must fill. Doc05's `with_structured_output(Model)` calls this API underneath. In Doc11 and Projects 4/13, a supervisor's routing decision and a reviewer's findings are both structured objects — the glue that lets one agent's output become safe input for the next agent's code.

Quick cheat sheet:

- Prompt "reply in JSON" = a wish. JSON mode = valid JSON. Structured outputs = your exact schema.
- `client.chat.completions.parse(..., response_format=MyModel)`, then read `message.parsed`.
- Always check `message.refusal` first.
- `Literal[...]` for categories, `X | None` for "might not be there."
- Shape is guaranteed; truth is not — validate important values yourself.

Error types, and which Doc02 rules apply to each

When a call fails, the `openai` library raises a named exception instead of handing you a raw status code — and each name is exactly one of Doc02's HTTP status groups, just with an SDK-specific name. `AuthenticationError` **is** a 401. `RateLimitError` **is** a 429. `APITimeoutError` is Doc02's timeout, arriving as a class instead of an exception you raised yourself. They all inherit from one base, `openai.APIError` — the same "one base class" idea Doc01's errors topic taught — so catch the specific ones first, a broad one last, and never a bare `except:`.

How it really works

- Doc02's rule applies unchanged: **retry temporary failures with backoff, never retry permanent ones.** 401/403/404/400 are permanent — retrying sends the same broken request again. 429/timeout/connection-error/5xx are temporary — worth a backoff retry.

- The client already retries connection errors, timeouts, 408, 409, 429 and 5xx **2 times** by default, with backoff — this is Doc02's `request_with_retry` loop, already built in. Set it on purpose: `OpenAI(timeout=30.0, max_retries=3)`.
- `RateLimitError` (429) usually means "slow down," but `e.code == "insufficient_quota"` means the account is out of credit — that one is permanent, and retrying it forever wastes time for nothing.
- `BadRequestError` with `e.code == "context_length_exceeded"` is Doc03's context-window limit, arriving as an exception — don't retry the same request, trim the history and send a new one.
- Order matters in `except` chains: `APITimeoutError` is a child class of `APIConnectionError`, so the specific one must come first or it's never reached.
- **Stacking two retry systems is the same mistake Doc02 warned about**: keep the client's default `max_retries=2` *and* wrap the call in your own retry loop, and one real failure can turn into up to 3×3 requests, with your own logs only showing your own attempts.

Error	HTTP status	Temporary or permanent?	What to do
`AuthenticationError`	401	Permanent	Don't retry — check `OPENAI_API_KEY`, exit cleanly
`BadRequestError`	400	Permanent for this request	Don't retry as-is — fix the parameter, or trim history if `context_length_exceeded`, then send a new request
`RateLimitError`	429	Usually temporary	Retry with backoff — **unless** `code == "insufficient_quota"`, which is permanent
`APITimeoutError` / `APIConnectionError`	(no status)	Temporary	Retry with backoff, up to a limit
`InternalServerError`	5xx	Temporary	Retry with backoff

Common mistakes:

- *Mistake:* keeping the client's default retries **and** wrapping every call in your own retry loop too. → *Symptom:* a single failure turns into far more real requests than your own logs show, and a failed call takes much longer than your backoff math predicts. → *Fix:* pick one layer — `OpenAI(max_retries=0)` with your own loop, or the library's retries with no loop of your own.
- *Mistake:* retrying every `RateLimitError` the same way. → *Symptom:* an out-of-credit account keeps retrying forever, burning time and never succeeding. → *Fix:* check `e.code == "insufficient_quota"` first — that one is permanent.

Where you'll meet it: Project 1 must handle a bad key, a rate limit, and a too-long conversation gracefully. Doc07's agent loop must survive one failed call among ten. Doc11's "error propagation

between agents" is this exact decision, made per agent: retry, skip, or stop the team. Doc12 turns these same errors into fallbacks and alerts; Doc14 drills a rate limit hitting mid-stream.

Quick cheat sheet:

- These error names **are** Doc02's status-code table, just with SDK-specific names.
- 401/403/404/400 → permanent, don't retry. 429/timeout/connection/5xx → temporary, retry with backoff.
- `insufficient_quota` on a `RateLimitError` is permanent, not temporary.
- The client already retries twice by default — set `timeout=` / `max_retries=` on purpose, and use only one retry layer.
- Catch specific errors first — `APITimeoutError` before `APIConnectionError`.

Chat Completions vs. the newer Responses API

OpenAI has two ways to talk to a model. **Chat Completions** (`client.chat.completions.create`) is the older, most widely copied one — you resend the full `messages` list every call and get back `choices`, exactly the pattern this whole document has used so far. The **Responses API** (`client.responses.create`) is newer — you send `input`, read `response.output_text`, and OpenAI can store the conversation on its own side, so you pass `previous_response_id` instead of resending everything yourself. Think of Chat Completions as carrying your own notebook to every meeting; Responses is leaving the notebook with the host and just saying "pick up where we left off."

How it really works

- Both are still an HTTP `POST` underneath — Doc02's request/response mechanics, timeouts and retries apply to either one; only the shape of the body and the reply changes.
- Server-stored history in Responses still bills the full history as input tokens on every call — the cost is identical to managing it yourself, per Doc03's "no memory, no free lunch" point; it just saves you the resending code.
- `instructions=` in the Responses API is **not** carried forward by `previous_response_id` — resend it every call if you want the rules to stick.
- Token usage field names differ: `response.usage.prompt_tokens` / `completion_tokens` (Chat Completions) vs. `response.usage.input_tokens` / `output_tokens` (Responses) — same Doc03 cost math, different key names to log.
- Structured output has a matching call: `client.responses.parse(..., text_format=Model)` → `response.output_parsed`, same idea as the Structured output topic above.
- Responses stores conversations by default — pass `store=False` where company data rules require it.
- The Assistants API (a third, older one) is being retired — don't start new work on it.

Situation	What to do	Why
Learning, Project 1, this curriculum's early docs	Chat Completions	You see and control the whole history yourself
Code that must also work with other providers or local models	Chat Completions	The most widely copied request shape
A new OpenAI-only agent app needing web/file search	Responses API	Built-in tools run on OpenAI's side
Want OpenAI to keep chat history for you	Responses API with `previous_response_id`	You send only the new message each call
Data must not be stored by the provider	Keep history yourself; if using Responses, `store=False`	Responses stores responses by default

Common mistakes:

- *Mistake:* calling `client.responses.create(...)` then reading `response.choices[0].message.content` (a Chat Completions habit). → *Symptom:* `AttributeError` — a Responses object has no `choices`. → *Fix:* check which API the line calls, and use the matching field (`output_text` for Responses).
- *Mistake:* assuming `previous_response_id` also carries the `instructions=` rules forward. → *Symptom:* the model quietly drops the system-level rules a few turns into a Responses-based conversation. → *Fix:* resend `instructions=` on every call, same as resending the `system` message in Chat Completions.

Where you'll meet it: Doc04 and Project 1 use Chat Completions throughout. Doc05's `ChatOpenAI` can use either API underneath. Doc06 and Doc07 support tool calling in both shapes. In Doc11 and Projects 4-5, most multi-agent frameworks sit on top of one of these two APIs — knowing both shapes means you can read any framework's logs and tell which one it's calling.

Quick cheat sheet:

- Chat Completions: `messages` in, `choices[0].message.content` out, you manage history.
- Responses: `input` in, `output_text` out, history can live on OpenAI's side via `previous_response_id`.
- Learn Chat Completions first — it shows you exactly how memory works, instead of hiding it.
- Don't mix the two response shapes in one piece of code.
- Assistants API is being retired — don't start new work on it.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- OpenAI API reference — the source of truth. Bookmark it, you'll come back often.
- OpenAI — Text generation / Chat Completions guide — the main call you'll use most.
- OpenAI — Function calling guide — read now to see the shape of it; full use starts in Doc06.
- OpenAI — Structured outputs guide — checked, guaranteed-shape answers.
- openai-python SDK (GitHub) — the README and `examples/` folder.
- openai-cookbook (GitHub) — real worked examples; skim the titles, read 2-3 that look useful.
- OpenAI — Responses API reference — the newer API.

Practice Exercises

Setup: same venv as before — if it's not active, `source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New package for this document: `pip install openai python-dotenv pydantic`.

Where your code lives: all of it under `04_openai_api/practice/` (`mkdir -p practice`), never loose beside this README — same convention as Doc01 and Doc02. None of this document's exercises share an underlying topic, so each one still gets its own file, but several build directly on Doc01's Build Task and Doc03's exercises — check the **Builds on** line under each one before starting from scratch.

The full file layout, all exercises:

```
practice/
├── chat_api_basics_practice.py     Basic
├── prompts/                        formal.txt, pirate.txt (Basic)
├── conversation_memory_practice.py Intermediate
├── streaming_practice.py           Real-world
├── structured_output_practice.py   Structured output
├── context_limit_practice.py       Edge cases
├── auth_and_cost_practice.py       Failure
└── build_task/                     Build Task — its own folder
    ├── main.py                     the terminal loop
    ├── chat_client.py              create_client(), send/stream_message()
    ├── schemas.py                  Pydantic model(s) for structured output
    ├── config.py                   copied from 01_python_foundations, unchanged
    ├── logging_setup.py            copied from 01_python_foundations, unchanged
    └── test_chat_client.py         proves the Test Cases
```

Why each script exists:

- `chat_api_basics_practice.py` — the single call every other document and project builds on.

- `prompts/` — the Basic exercise's system prompts as `.txt` files, read by `load_prompt()`.
- `conversation_memory_practice.py` — builds, by hand, the exact resend-everything behavior that *is* an LLM's "memory."
- `streaming_practice.py` — the difference between a chatbot that feels instant and one that feels like it's stalling.
- `structured_output_practice.py` — practices the Pydantic form (`str | None`, `Literal`, `model_validate`), the `.parse()` call, and an `/extract` command loop before the Build Task needs them.
- `context_limit_practice.py` — sees the real exception type/message once, instead of just reading about it.
- `auth_and_cost_practice.py` — catching the specific auth error is what makes a config problem fixable, not hidden.
- `build_task/chat_client.py` — the one client every later exercise/project in this app actually calls.
- `build_task/schemas.py` — makes "give me JSON" an actually-checked guarantee, not a hope.
- `build_task/main.py` — wires config, logging, and chat_client together into a real terminal app.
- `build_task/config.py` / `build_task/logging_setup.py` — copied unchanged from Doc01, so every later document imports the same settings/logging behavior.
- `build_task/test_chat_client.py` — catches a regression here before Project 1 (or Doc05/Doc06, which build on this same shape) hits it downstream.

Jump to an exercise: Basic · Intermediate · Real-world · Structured output · Edge cases · Failure · Build Task

Basic — your first real API call

- **What:** one call to the model, print the reply, then change the system prompt and watch the answer change. Last, load each system prompt from a `.txt` file with `load_prompt(name)` (`.read()`, `.lower()`).
- **Why:** this is the single call every other document and project builds on — get it working with your own eyes before adding anything else.
- **How to code it:** `client = OpenAI()`, then `client.chat.completions.create(model="gpt-4o-mini", messages=[{"role": "system", ...}, {"role": "user", ...}])`, and print `response.choices[0].message.content`. Change only the system message and compare. Then write `load_prompt(name)`, which opens `prompts/<name>.txt` (using `name.lower()`) with `with open(...)` and returns `.read()`.
- **Save as:** `practice/chat_api_basics_practice.py`.

- **Builds on:** Doc01's Build Task `config.py` / `load_config()` for `OPENAI_API_KEY`, and `get_logger(__name__)` for logging the reply — reuse both, don't rewrite them.
- **Used later by:** the Intermediate exercise right below, which wraps this same call in a loop.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — build real memory

- **What:** a loop that sends multiple turns, adding each one to a growing message list.
- **Why:** this *is* what "memory" means for an LLM — there's no other kind. Building it yourself once means you'll never be confused by it again.
- **How to code it:** start a `history` list with the system message. In a `while True` loop: read the user's line, `append` it, call the model with the whole `history`, print the reply and `append` it too; stop on `/quit`. Then pull the call into a typed `get_reply()` function.
- **Save as:** `practice/conversation_memory_practice.py`.
- **Builds on:** the Basic exercise's single call, and Doc03's "no memory" topic — you are building, by hand, the exact resend-everything behavior that topic described.
- **Used later by:** the Real-world exercise below, same loop with `stream=True` swapped in; the Build Task's `chat_client.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — make it feel alive with streaming

- **What:** switch the same call to `stream=True` and print words as they arrive instead of waiting for the whole reply.
- **Why:** this is the difference between a chatbot that feels instant and one that feels like it's stalling — a real, user-facing quality difference, not just a technical detail.
- **How to code it:** add `stream=True` to the same call, then `for chunk in stream:` print `chunk.choices[0].delta.content` with `end="", flush=True` whenever it isn't `None`, and add each piece to a `full_reply` string you return at the end.
- **Save as:** `practice/streaming_practice.py`.
- **Builds on:** the Intermediate exercise's message loop — same `history`, same call, just `stream=True` and a `for chunk in ...` loop instead of one return value.
- **Used later by:** the Build Task's `stream_message()`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Structured output — a checked answer, not a hope

- **What:** write a Pydantic model (`name: str | None`, `intent: str | None`, `urgency` as a `Literal`), check a good and a bad dict by hand with `model_validate()`, then let the model fill the same form with `.parse()`. Check `refusal` before `parsed`, and return `None` when

nothing useful was found. Last, read an `/extract <text>` command in a small loop with `.startswith()` and `.removeprefix()`.

- **Why:** the Structured output topic explains the idea; this is where you use it once, with your own eyes, before the Build Task and Project 1 depend on it.
- **How to code it:** write `class ExtractedInfo(BaseModel)`, check one good and one bad dict with `ExtractedInfo.model_validate(...)` inside `try` / `except ValidationError`, then call `client.beta.chat.completions.parse(..., response_format=ExtractedInfo)`. Read `.refusal` first, then `.parsed`, and return `None` when there's nothing usable. Last, wrap it in an `/extract` command inside a small loop.
- **Save as:** `practice/structured_output_practice.py`.
- **Builds on:** the Basic exercise's single call, and Doc02's JSON topic — `model_validate()` is the manual shape-check Doc02 did by hand, done by Pydantic.
- **Used later by:** the Build Task's `schemas.py` and `get_structured_reply()`; Project 1's triage agent; Doc05's `.with_structured_output()`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — what actually happens when you go over the limit

- **What:** force a too-long input on purpose and see the exact error OpenAI gives you.
- **Why:** reading about "context limit errors" is not the same as seeing the real exception type and message once, with your own eyes.
- **How to code it:** build a huge message (`"word " * 200_000`) and send it inside `try` with `except Exception as e:` first, to see what really comes back. Then catch `openai.BadRequestError` specifically and print its message.
- **Save as:** `practice/context_limit_practice.py`.
- **Builds on:** Doc03's Real-world exercise (guessing how many turns fit in a context window) — this is that same arithmetic, now verified against a real `BadRequestError` instead of a guess.
- **Used later by:** the Build Task's requirement to catch a too-long conversation gracefully.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — a bad key, and a cost comparison

- **What:** use a wrong API key on purpose and catch `AuthenticationError` specifically. Then, separately, compare token counts between a short system prompt and a long one, for the same task.
- **Why:** a bare `except` here would hide a config problem from you and your users forever — catching the specific error is what makes it fixable. The cost comparison is Doc03's Intermediate exercise, run for real this time.
- **How to code it:** create `OpenAI(api_key="sk-fake-key-123")` and make a call inside `try` / `except openai.AuthenticationError`. Then send the same user message twice — once

with a short system prompt, once with a long one — and compare `response.usage.prompt_tokens`.

- **Save as:** `practice/auth_and_cost_practice.py`.
- **Builds on:** Doc01's custom-error-class habit (catch the specific thing, not `Exception`), and Doc03's Cost topic — same formula, now driven by real `response.usage` numbers instead of hand math.
- **Used later by:** the Build Task's requirement to handle a bad key without a scary stack trace.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Project 1: Beginner LLM App

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: a terminal chatbot with real conversation memory, and one structured-output feature.

Requirements:

- A multi-turn conversation loop in the terminal, using the config/logger from Doc01.
- Uses Doc02's `http_client` style where it makes sense (or the library's own retries — pick one, and be able to explain why).
- At least one mode that gives back a checked, Pydantic-shaped answer (like "pull these fields out of what I just said").
- Handles gracefully: a bad key, a rate limit, a too-long conversation, a bad structured-output reply.

Inputs: whatever the user types into the terminal. **Outputs:** streamed replies in the terminal; a checked, structured object for the structured-output feature. **Constraints:** no secrets in code; full type hints; must not crash on any of the 4 problems below — it should handle each one gracefully, with a clear message.

```
04_openai_api/practice/build_task/
├── main.py              the terminal loop
├── chat_client.py       create_client(), send_message(), stream_message()
├── schemas.py           Pydantic model(s) for structured output
├── config.py            copied from 01_python_foundations's Build Task
├── logging_setup.py     copied from 01_python_foundations's Build Task
└── test_chat_client.py  proves the Test Cases below
```

- `main.py` — **What/Why:** wires config, logging, and chat_client together into a real terminal app — the file you actually run.
- `chat_client.py` — **What/Why:** the one client every later exercise/project reusing this app calls instead of hitting `openai` directly.

- `schemas.py` — **What/Why:** makes "give me JSON" an actually-checked guarantee, not a hope.
- `config.py` / `logging_setup.py` — **What/Why:** copied unchanged from Doc01, so this app uses the exact same settings/logging behavior as every other document.
- `test_chat_client.py` — **What/Why:** catches a regression here before Project 1, Doc05, or Doc06 (which all build on this same shape) hit it downstream.

Run it: `cd practice/build_task && python test_chat_client.py` — from inside the folder, so `from config import load_config` finds the file next to it.

Builds on: Doc01's Build Task `config.py` / `load_config()` and `get_logger(name)` — **copy** both in unchanged, same names, not a fresh explanation. Doc02's `http_client.py` retry pattern (or the library's own `max_retries=` — pick one, and write down why). All six practice exercises above — the first call, conversation memory, streaming, structured output, the context-limit error, and the auth-error/cost handling — fold into this one app; the structured-output exercise's `ExtractedInfo` and `extract_info()` become `schemas.py` and `get_structured_reply()`, and its command loop becomes `main.py`'s `/extract` branch.

Used later by: Project 1 is this same app, one level more serious — its own **Setup** section is what you actually follow to build the graded version; this Build Task is the practice run that removes the surprises first. Doc05 rewrites this same chatbot with LangChain; Doc06 adds tool calls to this same `chat_client.py` shape.

Functions/Components to build:

- `chat_client.py` → `create_client()`, `send_message(history, user_input) -> str`, `stream_message(history, user_input)`
- `schemas.py` → at least one Pydantic model for the structured-output feature
- `main.py` → the terminal loop, connecting everything together

Expected Behavior

- The conversation remembers earlier messages within one session (not across restarts — that's Doc09's job).
- A bad key → a clear error message, the program exits cleanly, no scary stack trace shown to the user.
- A too-long conversation → caught and reported, not an unhandled crash.
- A structured-output request → either a valid, checked object, or a clear "couldn't understand that" message — never a silent wrong answer.

Test Cases

Scenario	Expected
A normal 3-turn conversation	Each reply reflects the earlier turns
A wrong API key	`AuthenticationError` caught, clean exit
Input designed to go over the context limit	Handled, user is told, no crash
Structured-output request with a clear input	A valid Pydantic object comes back
Structured-output request with an unclear input	Either a best-effort object, or a clear failure — your choice, and write down why

Break-It / Debug Preview

- A message list that keeps growing until it quietly goes over the context limit in the middle of a conversation.
- Hitting a rate limit in the middle of a streamed reply. Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Chat Completions vs. Responses API · streaming vs. non-streaming trade-offs · how structured output is actually enforced · why "memory" is just resent history.

□ You Can Now Build Project 1

Docs 01-04 are everything Project 1 needs. Go to project_1_supportdesk_chat_and_triage/ and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 1 runs start to finish, handles at least 3 of the problems above gracefully, and you can explain every line without help. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** through **Hint 4**. Ask for the full solution only if you say **"Show me the solution."**